Autenticação em JavaScript

Parte 1 — Login · Criptografia de Senha · JWT

JS JAVASCRIPT | EXPRESS | JWT | BCRYPT

Made with GAMMA

O que é Autenticação?

Antes de proteger qualquer rota, é essencial entender os três conceitos fundamentais de segurança em aplicações web.

🔑 Autenticação

Verificar **QUEM** é o usuário

Exemplo: Login com email e senha

🛡️ Autorização

Verificar **O QUE** ele pode fazer

Exemplo: Admin pode deletar, usuário comum não

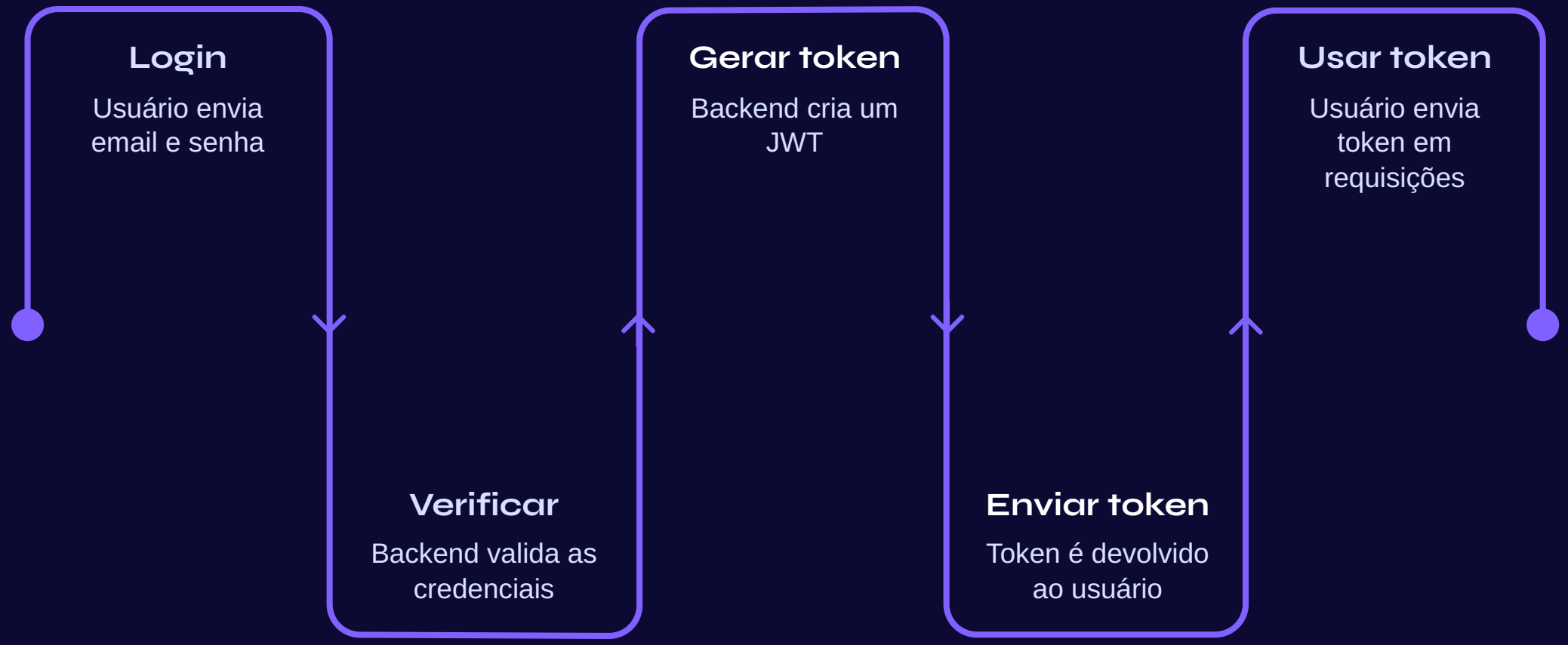
🎫 Token

Crachá digital do usuário

Exemplo: JWT gerado após o login

Made with GAMMA

Como o fluxo funciona



Este fluxo representa o ciclo completo de autenticação baseada em token — do login até o acesso às rotas protegidas.

Made with GAMMA

Criptografia de Senha com bcrypt

⚠️ Atenção

NUNCA salve senhas em texto puro no banco de dados

Instalação e uso

```
npm install bcryptjs

// Criptografar senha (ao criar usuário)
const hash = await
bcrypt.hash("minha123", 10);

// Verificar senha (no login)
const ok = await
bcrypt.compare("minha123", hash);
// true
```

💡 O número `10` é o custo do hash. Quanto maior, mais seguro e mais lento. **10 é o padrão recomendado.**

Made with GAMMA

JSON Web Token — 3 partes

Um JWT é composto por três partes separadas por pontos, cada uma com uma função específica:

🔵 HEADER

Tipo do token e algoritmo usado

🟢 PAYLOAD

Dados do usuário: id, role, email

🔴 ASSINATURA

Garante que o token não foi alterado

⚠️ O payload **NÃO é criptografado** — nunca coloque senhas nele. Mas é **assinado**, portanto não pode ser falsificado.

Made with GAMMA

Gerando e Verificando o JWT

Gerar token

```
const token = jwt.sign(
  {
    id: usuario.id,
    role: usuario.role
  },
  process.env.JWT_SECRET,
  { expiresIn: "1d" }
);
```

Verificar token

```
const decoded = jwt.verify(
 token,
 process.env.JWT_SECRET
);
// decoded = { id: 1, role: "admin" }
```

Made with GAMMA

Estrutura de Arquivos no Express

Onde fica cada arquivo?

```
back-end/
├── controllers/
│   └── authController.js
├── services/
│   └── authService.js
├── routes/
│   └── authRoutes.js
├── middlewares/
│   └── authMiddleware.js  ← NOVO
├── utils/
│   └── jwt.js             ← NOVO
└── server.js
```

💡 Dica importante

O **middleware** roda **ANTES** do controller — ele é a **porta de segurança** da sua API.

Made with GAMMA

authService.js e authController.js

authService.js

```
async login({ email, senha }) {
  const usuario =
    UsuarioService.buscarPorEmail(email);
  if (!usuario) return null;

  const ok = await bcrypt.compare(
    senha, usuario.senha
  );
  if (!ok) return null;

  const token = gerarToken({
    id: usuario.id,
    role: usuario.role
  });
  return { token, usuario };
}
```

authController.js

```
async login(req, res) {
 const resultado =
 await AuthService.login(req.body);

 if (!resultado)
 return res.status(401).json({
 mensagem: "Credenciais inválidas"
 });

 res.status(200).json(resultado);
}
```

Made with GAMMA

Boas Práticas de Senha e Token

🔐 JWT_SECRET no .env

Guarde o `JWT_SECRET` no arquivo `.env` — nunca escreva direto no código

⏰ Expiração do token

Defina expiração do token (`expiresIn: "1d"`)

🚫 Sem senhas no payload

Nunca coloque senhas no payload do JWT

🔒 Criptografe sempre

Sempre criptografe a senha antes de salvar no banco

```
# .env
JWT_SECRET=sua_chave_super_secreta_aqui
```

Made with GAMMA

Resumo da Parte 1

O que aprendemos até aqui

✅ Autenticação

Autenticação verifica **QUEM** é o usuário

✅ bcrypt.hash

`bcrypt.hash(senha, 10)` → criptografa

✅ bcrypt.compare

`bcrypt.compare(digitada, hash)` → verifica

✅ jwt.sign

`jwt.sign({id, role}, segredo)` → gera token

✅ jwt.verify

`jwt.verify(token, segredo)` → valida token

➡️ Próximo

Proteção de rotas com **Middleware**

Made with GAMMA